

✓ Lab 08 - Implementing Vision Transformer for Classification

```
# Importing Necessary Libraries :
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader
import torch.nn as nn
import torch.optim as optim
from transformers import ViTModel, ViTConfig

import warnings
warnings.filterwarnings('ignore')

# Loading data :
transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=3),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, tr
test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, tr

train_dataset

Dataset CIFAR10
  Number of datapoints: 50000
  Root location: ./data
  Split: Train
  StandardTransform
Transform: Compose(
  Grayscale(num_output_channels=3)
  Resize(size=(224, 224), interpolation=bilinear, max_size=None,
antialias=True)
  ToTensor()
  Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
)

test_dataset

Dataset CIFAR10
  Number of datapoints: 10000
  Root location: ./data
```

```

Split: Test
StandardTransform
Transform: Compose(
    Grayscale(num_output_channels=3)
    Resize(size=(224, 224), interpolation=bilinear, max_size=None,
    antialias=True)
    ToTensor()
    Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

class CIFARViT(nn.Module):
    def __init__(self, num_classes=10):
        super(CIFARViT, self).__init__()
        # Using a smaller ViT configuration
        config = ViTConfig(
            image_size=224,
            patch_size=16,
            num_channels=3,
            hidden_size=192,
            num_hidden_layers=6,
            num_attention_heads=6,
            intermediate_size=768,
            num_labels=num_classes
        )
        self.vit = ViTModel(config)
        self.classifier = nn.Linear(config.hidden_size, num_classes)

    def forward(self, x):
        outputs = self.vit(pixel_values=x)
        logits = self.classifier(outputs.last_hidden_state[:, 0, :])
        return logits

model = CIFARViT(num_classes=10)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(f"Using device: {device}")

➡ Using device: cuda

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)

def train(model, dataloader, criterion, optimizer, scheduler, epochs=5):
    train_losses, train_accs = [], []

```

```

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    scheduler.step()
    epoch_loss = running_loss / len(dataloader)
    epoch_acc = 100 * correct / total
    train_losses.append(epoch_loss)
    train_accs.append(epoch_acc)

    print(f'Epoch {epoch+1}/{epochs}, Loss: {epoch_loss:.4f}, Accuracy: {epoch_acc:.2}')

return train_losses, train_accs

def visualize_predictions(model, dataloader, num_images=5):
    model.eval()
    images, labels = next(iter(dataloader))
    images, labels = images.to(device), labels.to(device)

    with torch.no_grad():
        outputs = model(images[:num_images])
        _, preds = torch.max(outputs, 1)

    plt.figure(figsize=(15, 3))
    for i in range(num_images):
        plt.subplot(1, num_images, i+1)
        img = images[i].cpu().permute(1, 2, 0).numpy()
        img = img * 0.5 + 0.5
        plt.imshow(img[:, :, 0], cmap='gray')
        plt.title(f'Pred: {class_names[preds[i]]}\nTrue: {class_names[labels[i]]}')
        plt.axis('off')
    plt.show()

print("starting training")
train_losses, train_accs = train(model, train_loader, criterion, optimizer, scheduler, ep

```

 starting training
 Epoch 1/50, Loss: 1.9130, Accuracy: 29.13%

Epoch 2/50, Loss: 1.6659, Accuracy: 39.56%
Epoch 3/50, Loss: 1.5262, Accuracy: 45.11%
Epoch 4/50, Loss: 1.3710, Accuracy: 50.93%
Epoch 5/50, Loss: 1.2457, Accuracy: 55.61%
Epoch 6/50, Loss: 1.2073, Accuracy: 57.24%
Epoch 7/50, Loss: 1.1996, Accuracy: 57.41%
Epoch 8/50, Loss: 1.2343, Accuracy: 56.03%
Epoch 9/50, Loss: 1.2536, Accuracy: 55.21%
Epoch 10/50, Loss: 1.2367, Accuracy: 56.08%
Epoch 11/50, Loss: 1.2001, Accuracy: 57.21%
Epoch 12/50, Loss: 1.1222, Accuracy: 60.18%
Epoch 13/50, Loss: 1.0156, Accuracy: 64.07%
Epoch 14/50, Loss: 0.8962, Accuracy: 68.18%
Epoch 15/50, Loss: 0.7748, Accuracy: 72.58%
Epoch 16/50, Loss: 0.7321, Accuracy: 74.28%
Epoch 17/50, Loss: 0.7442, Accuracy: 73.87%
Epoch 18/50, Loss: 0.8119, Accuracy: 71.24%
Epoch 19/50, Loss: 0.8779, Accuracy: 68.78%
Epoch 20/50, Loss: 0.9225, Accuracy: 67.24%
Epoch 21/50, Loss: 0.9170, Accuracy: 67.56%
Epoch 22/50, Loss: 0.8636, Accuracy: 69.28%
Epoch 23/50, Loss: 0.7719, Accuracy: 72.61%
Epoch 24/50, Loss: 0.6297, Accuracy: 77.88%
Epoch 25/50, Loss: 0.4884, Accuracy: 82.85%
Epoch 26/50, Loss: 0.4404, Accuracy: 84.91%
Epoch 27/50, Loss: 0.4492, Accuracy: 84.35%
Epoch 28/50, Loss: 0.5341, Accuracy: 81.02%
Epoch 29/50, Loss: 0.6290, Accuracy: 77.65%
Epoch 30/50, Loss: 0.6912, Accuracy: 75.24%
Epoch 31/50, Loss: 0.7098, Accuracy: 74.51%
Epoch 32/50, Loss: 0.6588, Accuracy: 76.27%
Epoch 33/50, Loss: 0.5387, Accuracy: 80.69%
Epoch 34/50, Loss: 0.3666, Accuracy: 87.15%
Epoch 35/50, Loss: 0.2174, Accuracy: 92.88%
Epoch 36/50, Loss: 0.1638, Accuracy: 95.14%
Epoch 37/50, Loss: 0.1761, Accuracy: 94.36%
Epoch 38/50, Loss: 0.2797, Accuracy: 90.09%
Epoch 39/50, Loss: 0.4009, Accuracy: 85.27%
Epoch 40/50, Loss: 0.4836, Accuracy: 82.45%
Epoch 41/50, Loss: 0.5033, Accuracy: 81.73%
Epoch 42/50, Loss: 0.4556, Accuracy: 83.54%
Epoch 43/50, Loss: 0.3174, Accuracy: 88.79%
Epoch 44/50, Loss: 0.1494, Accuracy: 95.01%
Epoch 45/50, Loss: 0.0539, Accuracy: 98.69%
Epoch 46/50, Loss: 0.0303, Accuracy: 99.60%
Epoch 47/50, Loss: 0.0335, Accuracy: 99.38%
Epoch 48/50, Loss: 0.1542, Accuracy: 94.48%
Epoch 49/50, Loss: 0.2517, Accuracy: 90.83%
Epoch 50/50, Loss: 0.3317, Accuracy: 88.04%

